

Serviço Nacional de Aprendizagem Comercial
Centro Universitário Senac-RS
Curso Superior de Tecnologia em Redes de Computadores

Unisenac
Centro Universitário RS

Senac Fecomércio
Sesc

JAMES JOSÉ GONÇALVES

STREAMING DE VÍDEO E ANÁLISE DE PROTOCOLOS

Pelotas
2025

RESUMO

Nesta prática, realizei a configuração e análise de diferentes tipos de streaming de vídeo em um ambiente de laboratório. O objetivo principal foi entender na prática as diferenças entre os modos de transmissão Unicast, Broadcast e Multicast utilizando o protocolo UDP. Para isso, utilizei as ferramentas VLC Media Player e OBS Studio como transmissores e receptores. Além da configuração dos softwares, precisei ajustar a infraestrutura de rede, criando rotas estáticas no Windows e no Linux para permitir o tráfego Multicast. Também testei outros protocolos importantes, como HTTP e RTSP, analisando o comportamento de cada um no Wireshark, desde o handshake até o fluxo de dados. Por fim, realizei um teste de estresse com a ferramenta Clumsy, comprovando a sensibilidade do protocolo UDP à perda de pacotes e latência.

Sumário

1. INTRODUÇÃO.....	3
2. DESENVOLVIMENTO.....	4
2.1 O que é um serviço de streaming de vídeo?.....	4
2.2 Transmissão de Vídeo via UDP no VLC.....	5
2.2.1 Unicast.....	10
2.2.2 Broadcast.....	13
2.2.3 Multicast.....	15
2.2.4 Vantagem do Envio em Multicast.....	18
2.3 Streaming utilizando servidor HTTP no VLC.....	19
2.4 Transmissão usando o protocolo RTSP no VLC.....	24
2.5 Transmissão Multicast com OBS Studio.....	27
2.5.1 Configuração da Fonte e Saída no OBS.....	28
2.5.2 Configuração de Rotas (Windows e Linux).....	30
2.5.3 Recepção do Streaming.....	31
2.5.4 Simulação de Falhas com Clumsy.....	32
2.5.5 Análise de Tráfego no Wireshark.....	34
3. CONCLUSÃO.....	36
4. REFERÊNCIAS.....	37

1. INTRODUÇÃO

O streaming de vídeo é a tecnologia que movimenta a maior parte do tráfego da internet hoje em dia, sendo a base para serviços como YouTube, Netflix, IPTV e videoconferências. O objetivo deste relatório é demonstrar o funcionamento técnico dessas transmissões, saindo da teoria e indo para a configuração prática.

A atividade foca em entender como o vídeo é transportado pela rede. Para isso, explorei os três principais modos de envio: Unicast (um para um), Broadcast (um para todos) e Multicast (um para um grupo), que é essencial para economizar banda em grandes transmissões.

Utilizei ferramentas: o VLC para testes rápidos, o OBS Studio para simular uma transmissão profissional de área de trabalho e câmera, e o Wireshark para inspecionar o que realmente acontece nos bastidores da rede. Também abordei a importância dos protocolos de transporte, comparando a confiabilidade do TCP (usado no HTTP) com a velocidade do UDP, e analisando como a infraestrutura de rede (rotas) impacta diretamente no sucesso da transmissão.

2. DESENVOLVIMENTO

2.1 O que é um serviço de streaming de vídeo?

Streaming de vídeo é quando assistimos um vídeo pela internet sem precisar baixar o arquivo inteiro antes. O conteúdo vai chegando em partes, em tempo real, e o player já começa a mostrar assim que recebe os primeiros pedaços. Isso é bem diferente do download normal, onde só dá pra abrir o arquivo depois que ele terminou de baixar.

Esse tipo de tecnologia é o que move praticamente tudo que a gente usa hoje em dia: YouTube, Netflix, TikTok, chamadas de vídeo, lives, IPTV e por aí vai. O vídeo pode ser gravado ou ao vivo, como lives ou webconferências.

O streaming consome bastante banda, principalmente vídeo. Hoje a maior parte do tráfego da internet é vídeo: um vídeo em 4K pode exigir 25 Mbps só pra manter a reprodução estável. Por isso as plataformas trabalham com diferentes qualidades (SD, HD, Full HD, 4K) e vão ajustando automaticamente conforme a banda disponível.

De forma técnica o streaming funciona usando protocolos e codecs. Os codecs “comprimem” o vídeo para facilitar o envio, e os protocolos definem como esse vídeo é transmitido. Os mais comuns hoje são HLS e DASH, que dividem o vídeo em pequenos trechos e adaptam a qualidade conforme a rede. Também existem protocolos focados em baixa latência, como QUIC, RTP/UDP e SRT, usados em transmissões ao vivo e em videoconferências.

Outra coisa importante é que vídeo em tempo real é muito sensível a atraso, perda de pacotes e variações na rede (jitter). Por isso, em ambientes profissionais, é normal configurar QoS para garantir prioridade desse tráfego.

Resumindo de um jeito simples: streaming de vídeo é como um “rio” de dados que nunca pode parar. Se a rede não der conta, o vídeo trava. Se estiver tudo estável, ele flui sem problemas.

2.2 Transmissão de Vídeo via UDP no VLC

Para fazer os testes, usei o VLC como servidor de streaming e configurei o envio de vídeo usando o modo UDP (Legacy). O processo inicial é sempre o mesmo: escolhi o arquivo de vídeo, adicionei um novo destino UDP e defini uma porta (usei a mesma porta nos três testes para manter o padrão). A única coisa que muda entre unicast, broadcast e multicast é o endereço que o pacote vai usar.

Depois dessa configuração básica, realizei os testes separando cada tipo de transmissão. Para esses testes, utilizei uma VM Windows 10 para realizar o streaming.

O procedimento padrão no VLC para as três formas de transmissão foi o seguinte:

- Baixei um vídeo do servidor "Mussum" no formato AVI para a transmissão, mas também fiz testes com a câmera de vídeo e a área de trabalho.
- No VLC, acessei o menu "Mídia" e cliquei na opção "Transmitir".
- Na janela "Abrir Mídia", utilizei a aba "Arquivo", adicionei o vídeo e cliquei no botão "Transmitir".

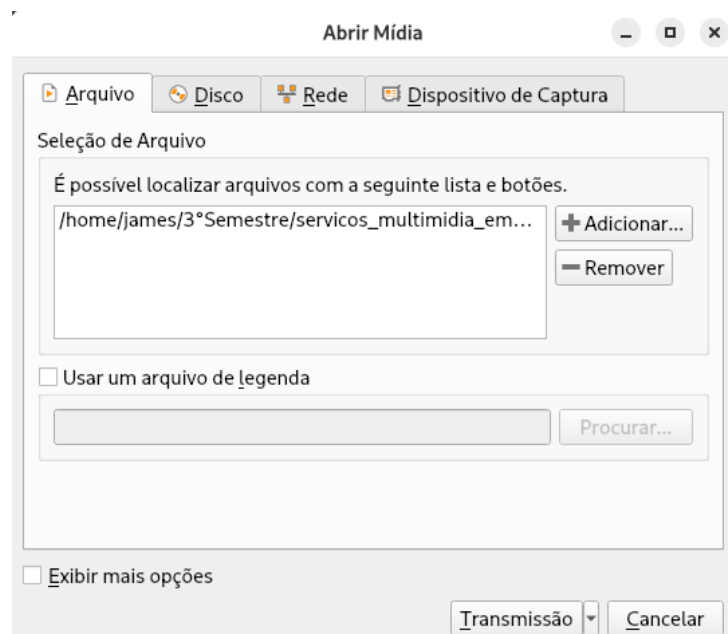


Figura 1: Adicionando o vídeo

Em seguida, na tela “Saída de Transmissão”, selecionei a opção “UDP (Legacy)” no menu “Novo Destino” e cliquei no botão “Adicionar”. Também marquei a opção “Exibir localmente” para conseguir assistir ao vídeo na minha máquina enquanto ele é transmitido.

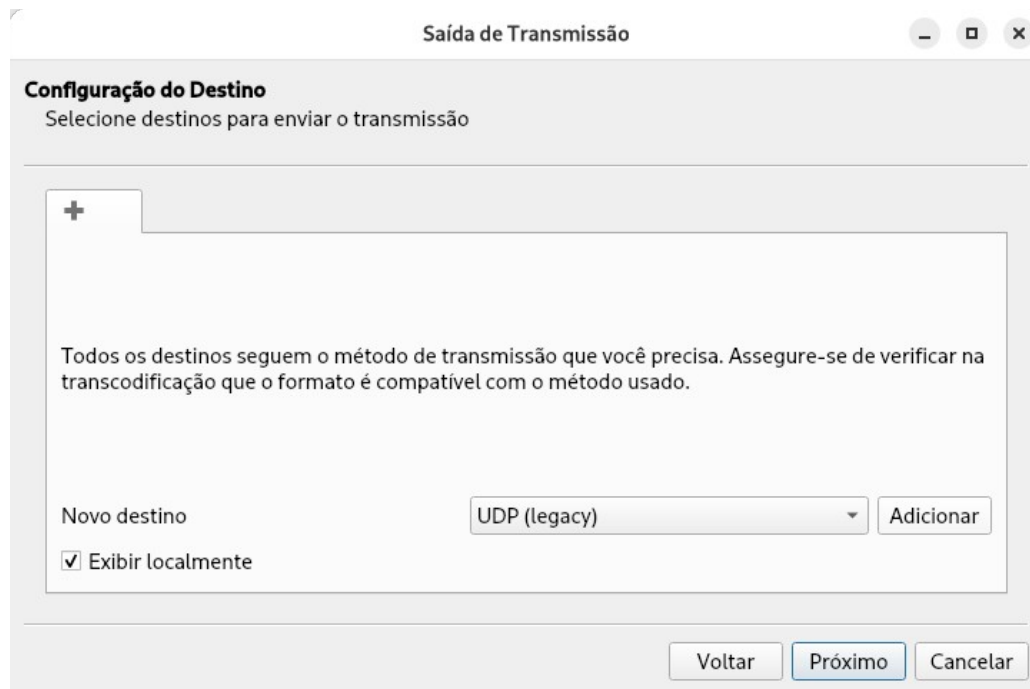


Figura 2: seleção UDP e a caixa Exibir Localmente marcada

O passo seguinte foi definir o endereço e a porta de acordo com o tipo de transmissão (unicast, broadcast ou multicast). Na imagem abaixo, mostro o exemplo da configuração que fiz para o Unicast, onde apontei diretamente para o IP da máquina de destino (172.16.0.231) usando a porta 4444.

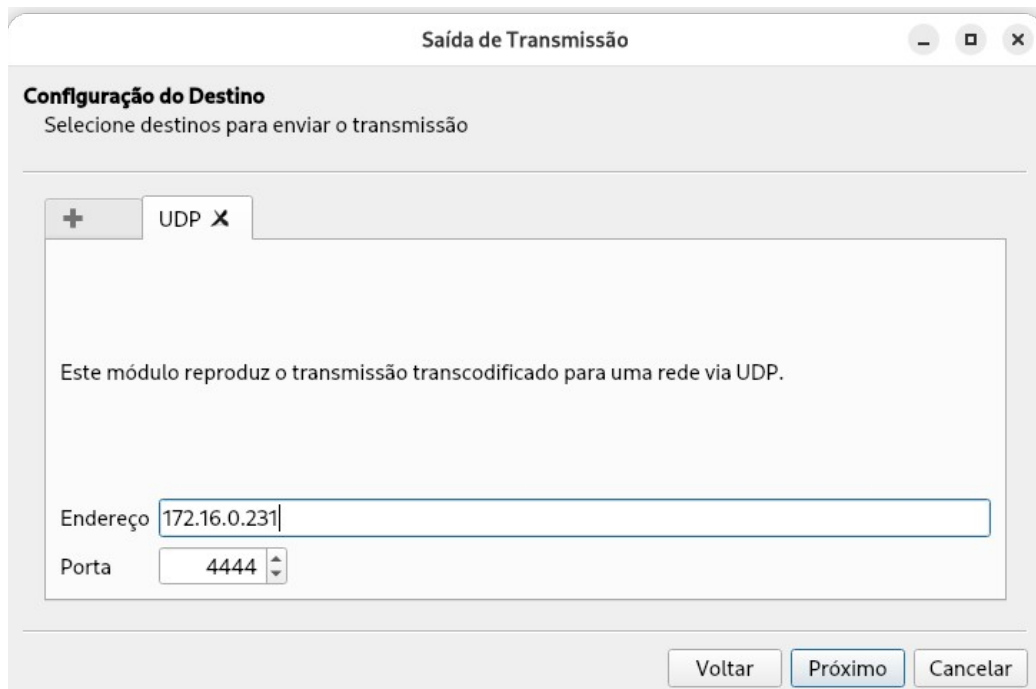


Figura 3: Exemplo de configuração

O passo seguinte na configuração foi definir as opções de transcodificação. Marquei a opção para ativar a transcodificação e escolhi o perfil padrão sugerido para a prática: “Video - MPEG-2 + MPGA (TS)”

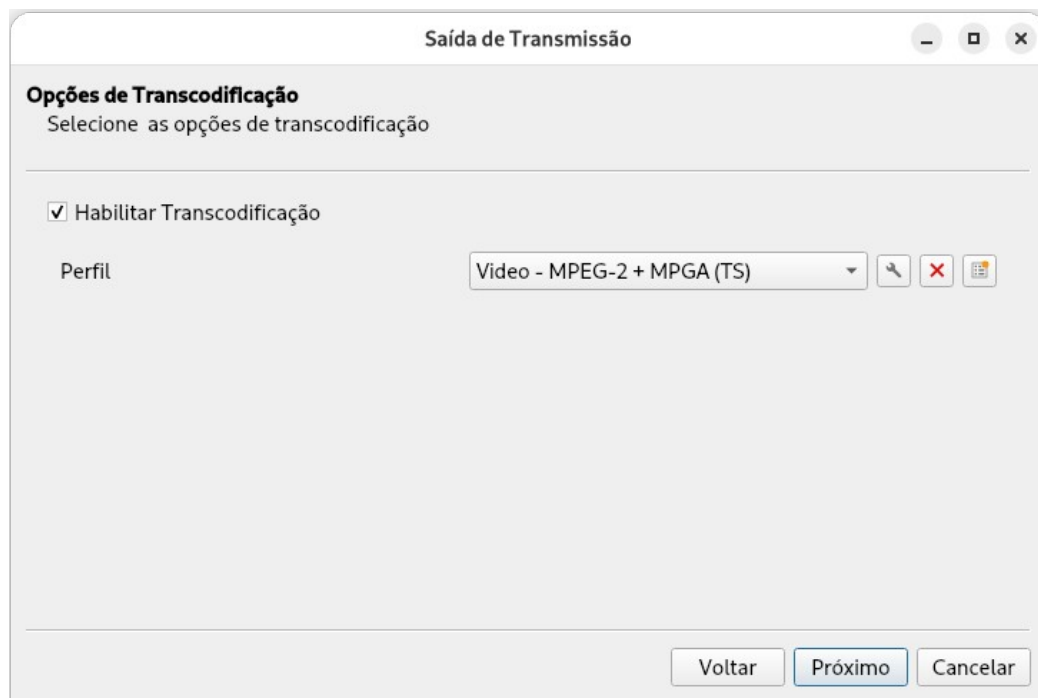


Figura 4: Transcodificação com o perfil MPEG-2 + MPGA (TS) selecionado

Na última etapa, em “Configuração de Opções”, mantive as opções padrão do VLC e cliquei no botão “Transmissão”. Isso iniciou o fluxo de vídeo para o endereço e porta que eu tinha definido.

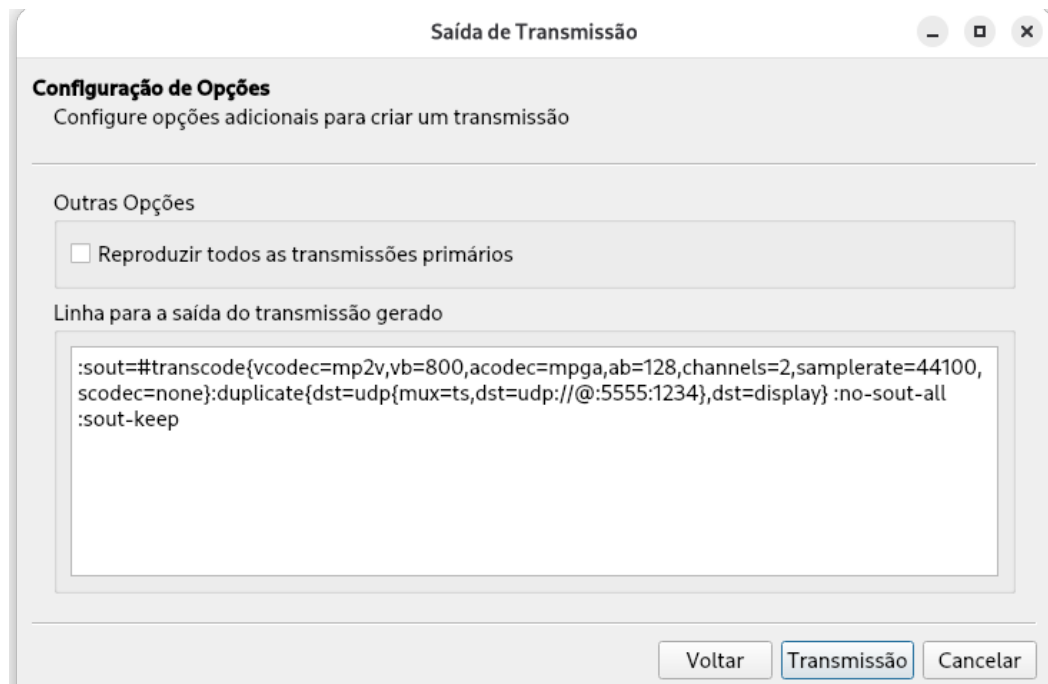


Figura 5: Configuração de Opções (padrão)

2.2.1 Unicast

No modo Unicast, o pacote de dados é enviado para apenas um destino específico. Eu configurei o VLC para enviar o fluxo de vídeo diretamente para o IP da máquina cliente. Para receber, o cliente VLC apenas precisa abrir o stream na porta definida. Conforme a imagem abaixo, na tela de saída de transmissão, configurei o destino UDP apontando para o endereço 172.16.0.231 e utilizando a porta 4444, o que iniciou a transmissão exclusiva para aquele host.

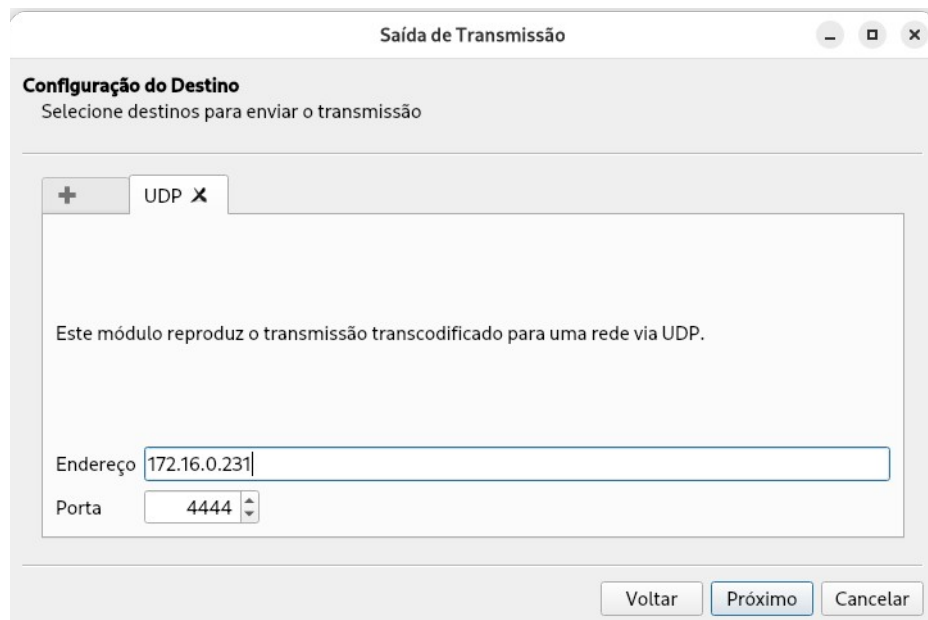


Figura 6: Configuração de destino UDP para Unicast

Para verificar se a transmissão estava chegando na máquina, utilizei o VLC no cliente. Fui na opção de abrir mídia de rede e inseri o endereço `udp://@:4444` para que o VLC escutasse o fluxo na porta definida.

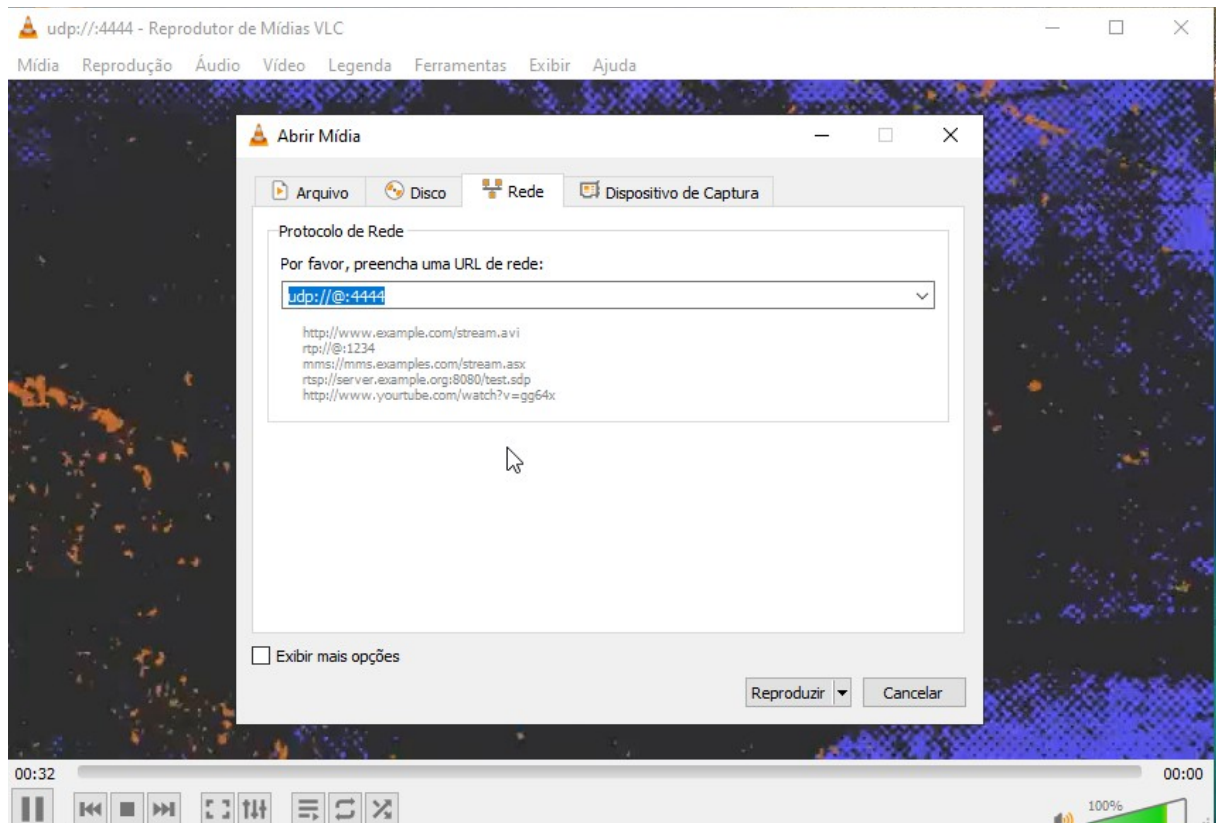


Figura 7: Configuração no VLC cliente para receber o stream unicast na porta 4444

Durante o teste em unicast, usei o filtro `udp.port == 4444` no Wireshark para capturar somente os pacotes da transmissão enviada pelo VLC. Com o filtro ativo, ficou claro que o streaming estava funcionando corretamente, porque os pacotes UDP apareciam chegando de forma contínua, sempre do IP do servidor para o IP do cliente. Esse comportamento é o esperado no unicast, já que a transmissão é direta de um host para outro, sem envolver a rede inteira. Só com esse print já dá para ver que o fluxo estava ativo e estável, confirmando a transmissão do vídeo em tempo real.

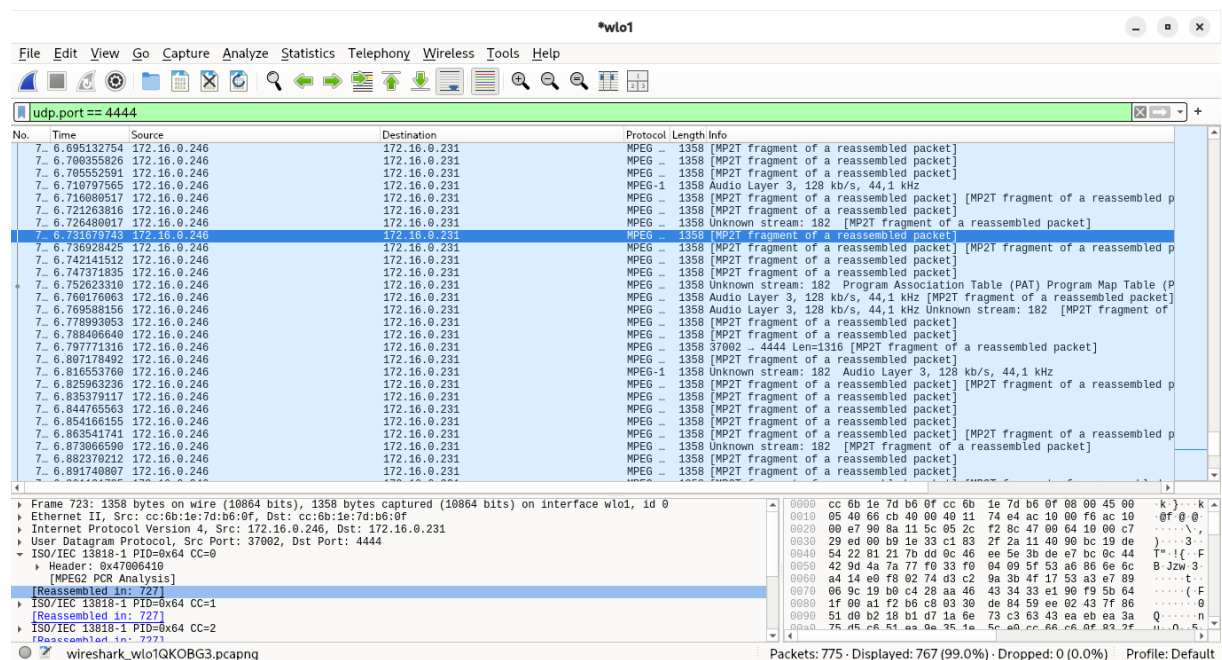


Figura 8: Captura do Wireshark confirmando o fluxo Unicast: pacotes UDP contínuos do servidor para o cliente na porta 4444

2.2.2 Broadcast

No modo Broadcast, configurei o VLC para enviar o fluxo de vídeo para o endereço de rede de broadcast, por exemplo 172.16.0.255. Nesse modo, todos os dispositivos da rede recebem o pacote, então qualquer VLC que estiver "escutando" na porta consegue reproduzir o stream.

Para verificar se a transmissão estava chegando na máquina cliente, utilizei o VLC. Fui na opção de abrir mídia de rede e inseri o endereço `udp://@:4444` para que o player escutasse o tráfego de broadcast na porta definida.

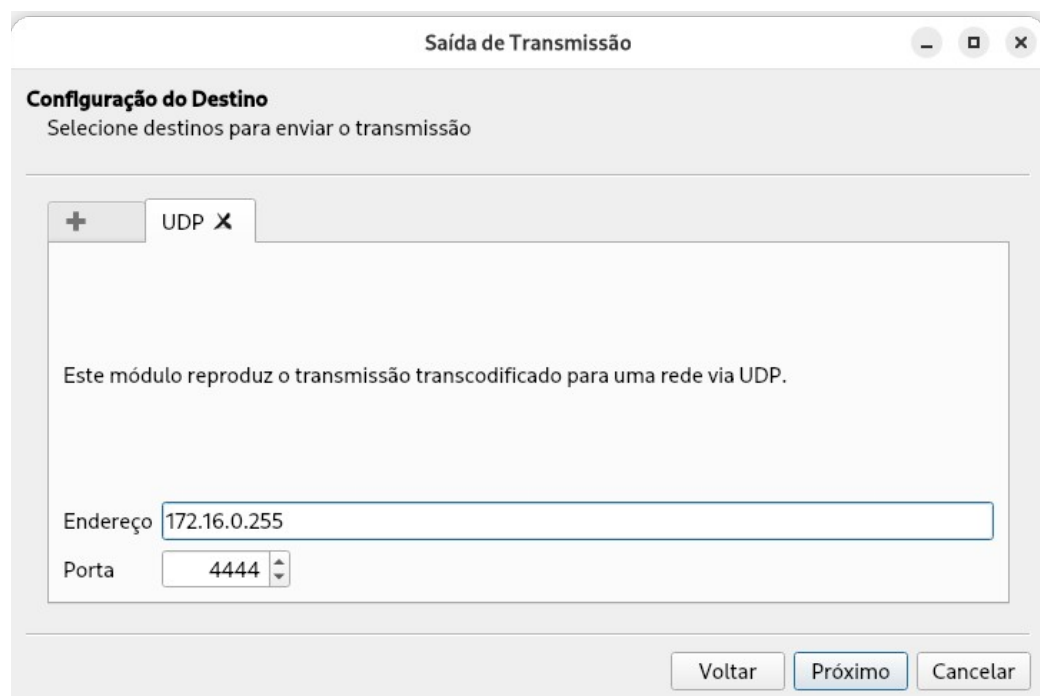


Figura 9: Configuração no VLC cliente para receber o stream de Broadcast na porta 4444

A recepção do stream no cliente foi configurada exatamente da mesma forma que no Unicast. Utilizei o VLC, fui na opção de abrir mídia de rede e inseri o endereço `udp://@:4444` para que o player escutasse o fluxo na porta definida.

A diferença principal é que, como o fluxo foi enviado para o endereço de broadcast (172.16.0.255), todos os dispositivos dentro do mesmo domínio de broadcast receberam a transmissão simultaneamente, e não apenas um host específico.



Figura 10: VLC cliente recebendo a transmissão de vídeo em modo Broadcast na porta 4444

Na transmissão em broadcast, consegui identificar claramente os endereços das camadas de enlace e de rede. No quadro Ethernet, o MAC de destino aparece como `ff:ff:ff:ff:ff:ff`, que é o endereço de broadcast. O MAC de origem é a interface que fez o envio. Já no cabeçalho IP, o endereço de destino é 172.16.0.255, que é o broadcast da rede, e o IP de origem é o da máquina que fez a transmissão (172.16.0.246). Com isso, ficou fácil confirmar que realmente se tratava de um envio broadcast pela rede.

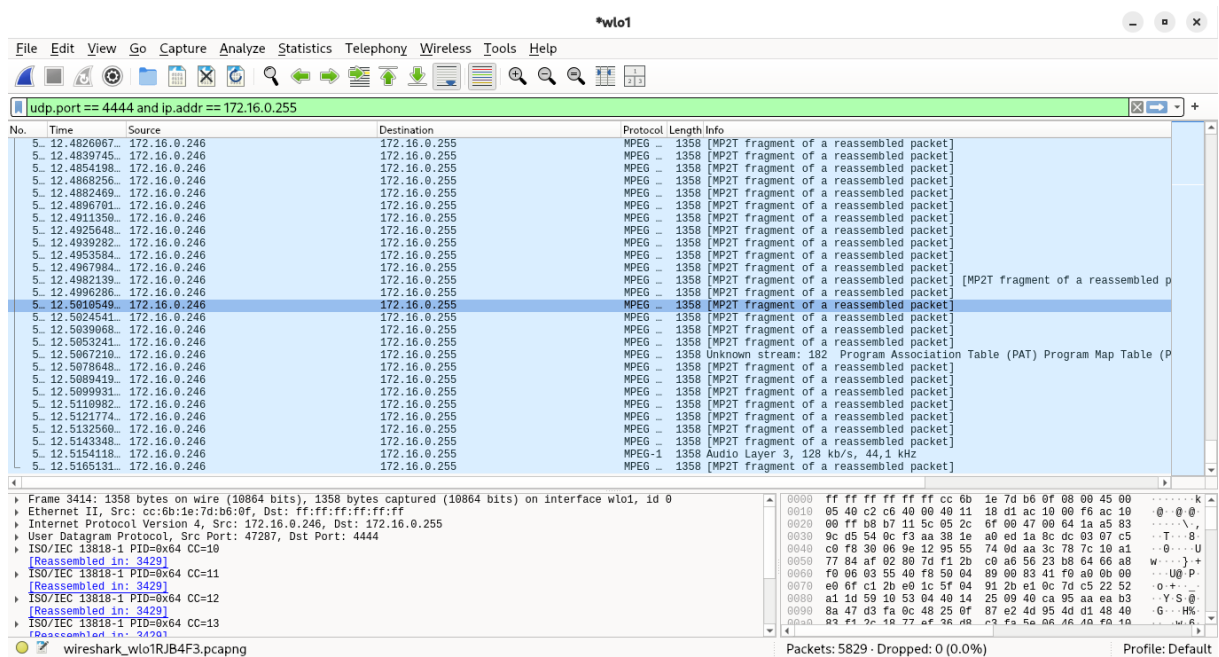


Figura 11: Captura do Wireshark do tráfego Broadcast

2.2.3 Multicast

No teste de Multicast, configurei o VLC para enviar o fluxo para um endereço dentro do range específico de multicast, usando o endereço 224.0.0.99. Esse método é mais eficiente do que o Broadcast, porque o pacote só é enviado para os dispositivos que se inscrevem no grupo.

Conforme a imagem, na tela de saída de transmissão, confirmei o destino UDP como o endereço de grupo 224.0.0.99 e a porta 4444 para iniciar a transmissão.

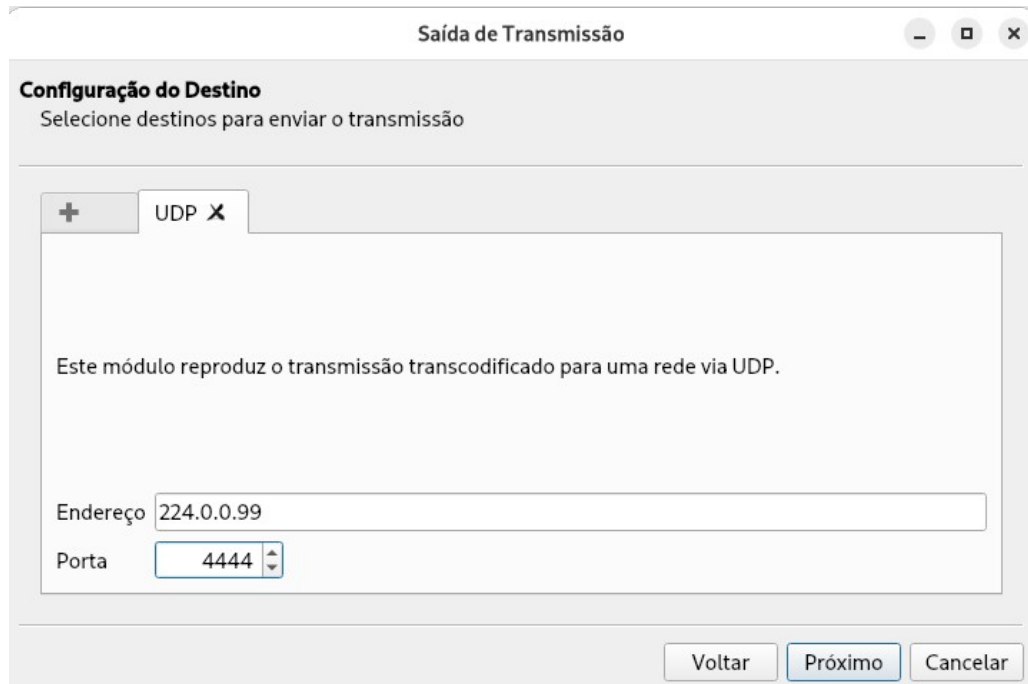


Figura 12: Configuração de destino UDP para Multicast

Para verificar se o fluxo estava sendo recebido, acessei o VLC no cliente. Usei a opção de abrir mídia de rede e inseri o endereço completo do grupo multicast: `udp://224.0.0.99:4444`. O VLC conseguiu abrir o stream e reproduzir o vídeo normalmente, confirmando a recepção do grupo.



Figura 13: VLC cliente recebendo e reproduzindo o stream Multicast do grupo 224.0.0.99

Para testar a transmissão em multicast, configurei no VLC o endereço 224.0.0.99 e comecei a captura no Wireshark usando o filtro `ip.addr == 224.0.0.99`.

Logo que a transmissão iniciou, apareceram vários pacotes saindo da máquina que estava enviando o vídeo (172.16.0.246) e indo para o grupo multicast (224.0.0.99). No print, fica claro que o tráfego está sendo enviado especificamente para esse endereço de multicast, o que confirma que o VLC realmente transmitiu no modo Multicast. Também dá para notar que os pacotes são do tipo MPEG-TS, mostrando que é o fluxo de vídeo sendo enviado em tempo real.

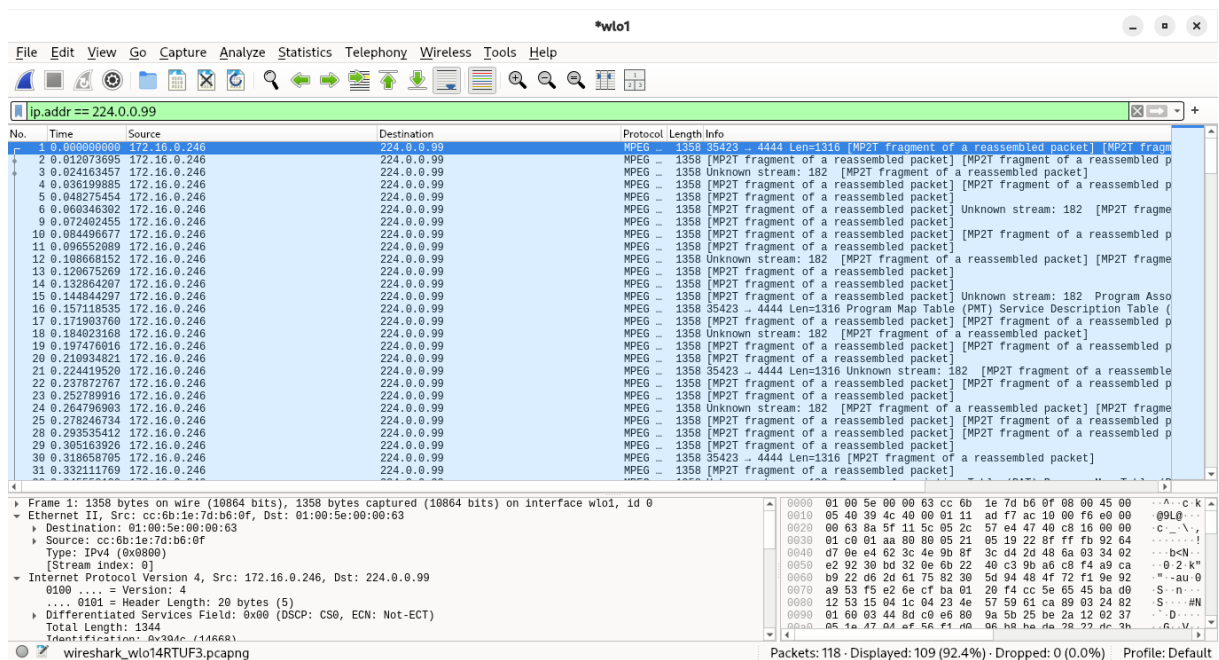


Figura 14: Captura do Wireshark do tráfego Multicast

2.2.4 Vantagem do Envio em Multicast

A principal vantagem do envio em Multicast é a eficiência na distribuição de conteúdo para múltiplos destinatários, sem sobrecarregar a origem da transmissão com fluxos individuais.

- **Otimização da Largura de Banda:** Em uma transmissão Multicast, a origem envia apenas um pacote para o endereço do grupo, mesmo que haja vários destinatários interessados. Se a mesma transmissão fosse feita em modo Unicast, a origem precisaria enviar uma nova transmissão para cada destinatário adicional.
- **Distribuição Inteligente (Escalabilidade):** O Multicast evita essa sobrecarga, sendo ideal para distribuir o mesmo conteúdo para múltiplos destinatários. O pacote só é replicado pelos dispositivos de rede (roteadores e switches) e entregue nos hosts que se inscreveram ativamente naquele grupo, economizando largura de banda da rede local.

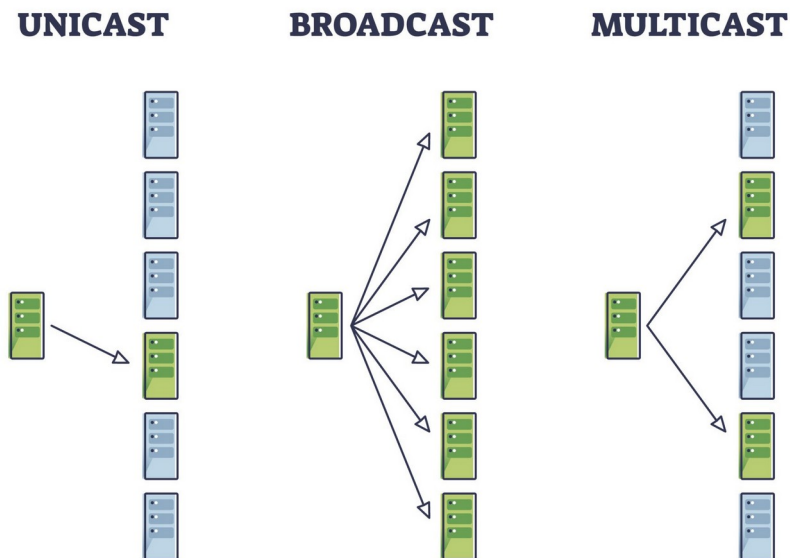


Figura 15: Comparação dos tipos de envio.

- Comparação com Broadcast: O Multicast é mais eficiente do que o Broadcast porque, embora ambos enviem o pacote para múltiplos hosts, o Multicast atinge apenas o subconjunto de hosts interessados (o grupo), enquanto o Broadcast inunda todos os hosts da rede ou da VLAN.
- Sincronização: Em aplicações em tempo real, como na prática de laboratório, o Multicast garante que todos os destinatários do grupo recebam o pacote de forma síncrona, permitindo que todos vejam o mesmo vídeo simultaneamente.

2.3 Streaming utilizando servidor HTTP no VLC

Além das transmissões em UDP (unicast, broadcast e multicast), também realizei o streaming usando o protocolo HTTP direto pelo VLC. Nesse modo, o próprio VLC funciona como um pequeno servidor web, e qualquer cliente pode acessar o vídeo colocando a URL no navegador ou no próprio VLC.

Para isso, criei uma nova transmissão no VLC, selecionei um arquivo de vídeo e escolhi o protocolo HTTP como saída. Depois defini uma porta e um caminho; no meu caso, usei a porta 8800 e coloquei o caminho /video1.

Com isso, o conteúdo ficou acessível pelo endereço:

http://IP_do_servidor:8800/video1

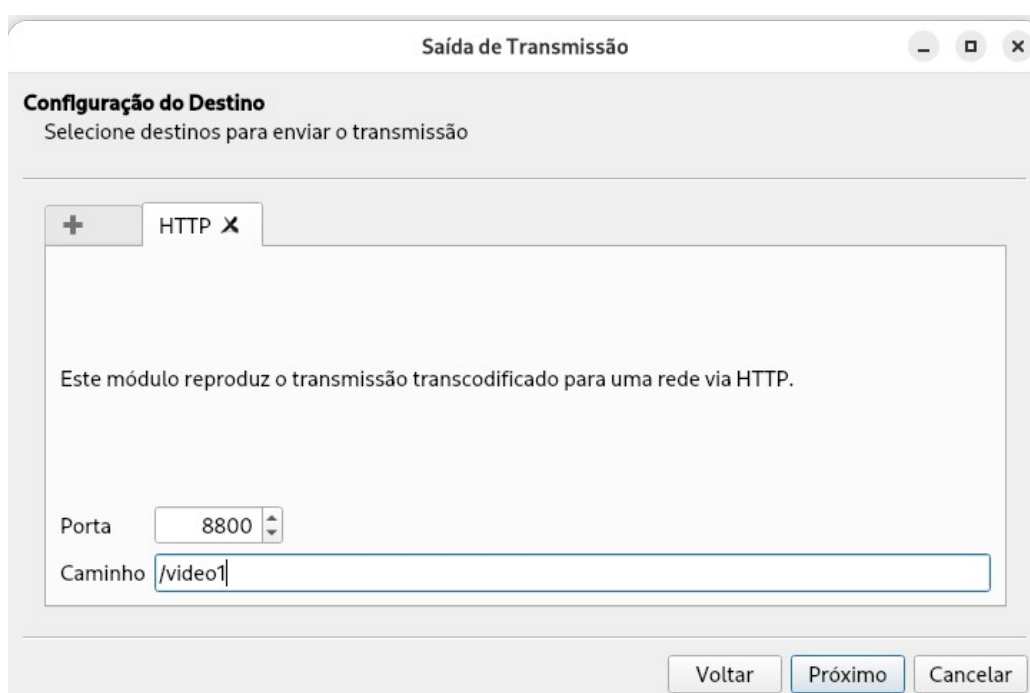


Figura 16: Configuração de destino do streaming via protocolo HTTP na porta 8800

Também ativei a transcodificação e escolhi o perfil Video - H.264 + MP3 (MP4), que é o recomendado no roteiro. Esse perfil é mais eficiente e garante boa compatibilidade com o protocolo HTTP.

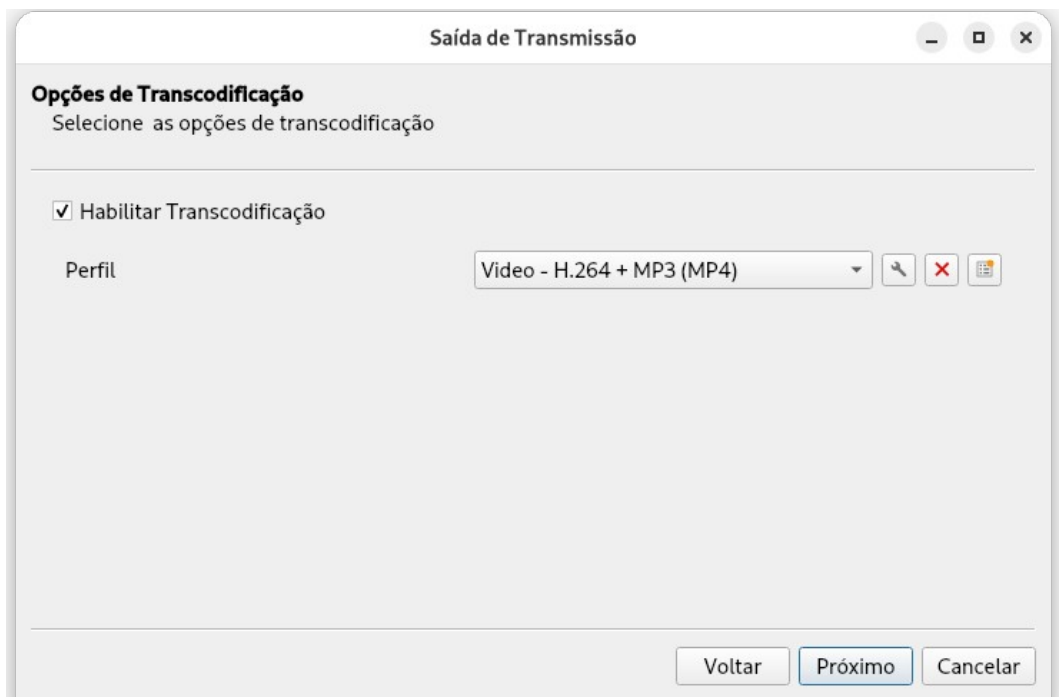


Figura 17: Ativando a transcodificação no VLC e selecionando o perfil H.264 + MP3 (MP4)

No lado do cliente, bastou abrir o VLC, acessar "Abrir Transmissão de Rede" e colocar a mesma URL do servidor (<http://172.16.0.246:8800/video1>). O vídeo carregou normalmente, mostrando que o streaming HTTP estava funcionando sem problemas.

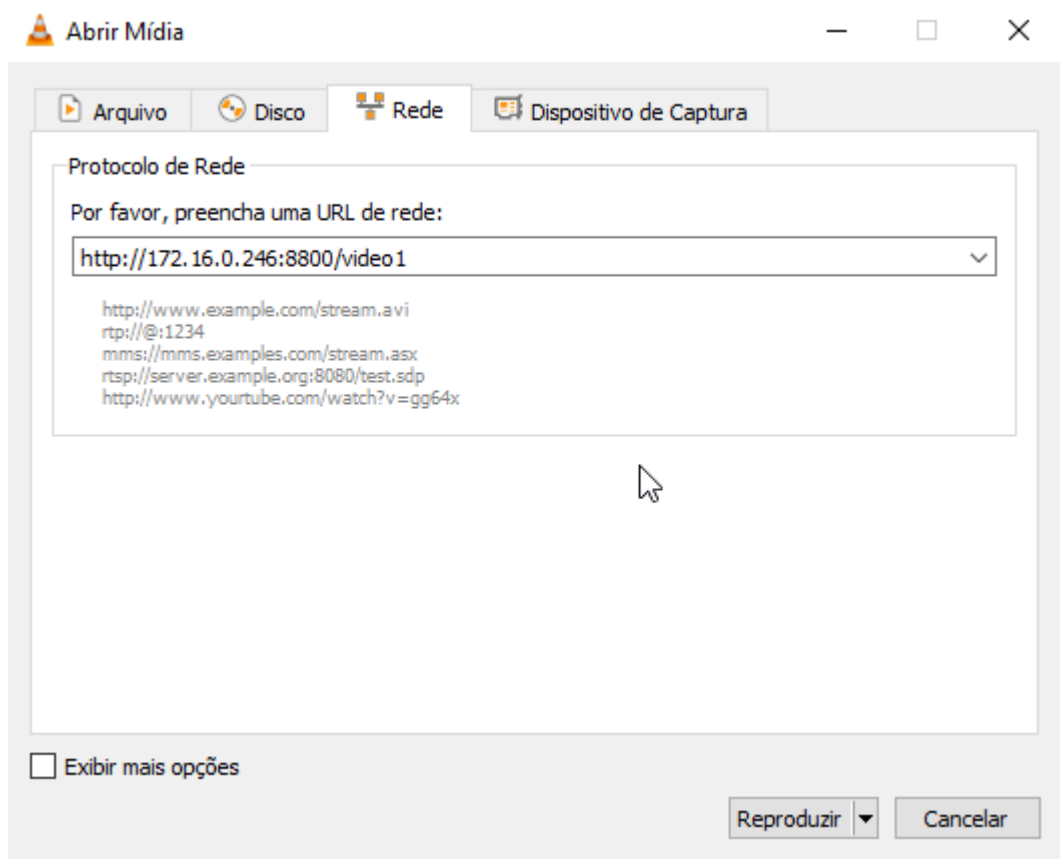


Figura 18: VLC cliente acessando o streaming via URL HTTP na porta 8800

Para analisar o tráfego do streaming via HTTP, iniciei a captura no Wireshark usando o filtro `tcp.port == 8800`, que foi a porta configurada no VLC. Assim que o cliente começou a receber o vídeo, vários pacotes TCP apareceram na tela. É possível ver claramente a comunicação entre o IP do cliente e o IP do servidor, sempre usando a porta 8800 no destino.

Os pacotes aparecem em sequência, com vários segmentos de 1460 bytes, o que é típico do transporte de vídeo pela rede quando o VLC está enviando o conteúdo via HTTP. Também aparecem pacotes ACK confirmando a entrega, já que o TCP

garante confiabilidade. Esse padrão de “PSH, ACK” seguido por novos segmentos mostra que o vídeo está sendo transmitido continuamente e que o cliente está recebendo o stream sem problemas.

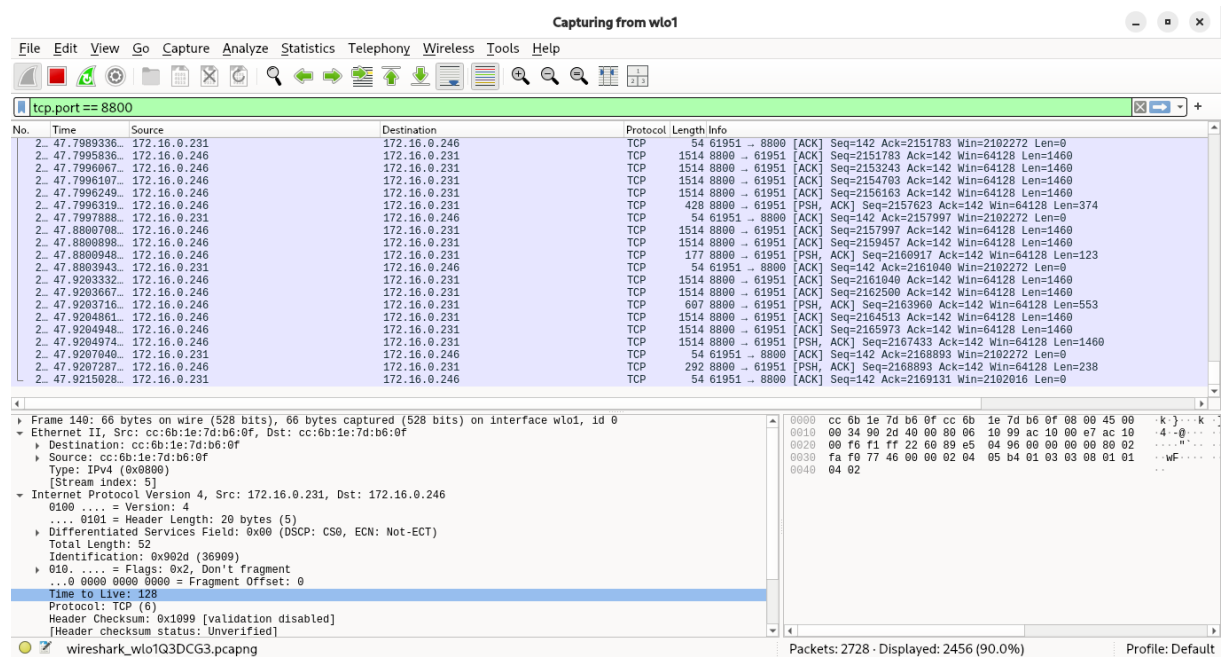


Figura 19: Captura do Wireshark do tráfego HTTP

2.4 Transmissão usando o protocolo RTSP no VLC

Além dos testes com UDP e HTTP, também realizei a transmissão usando o protocolo RTSP (Real-Time Streaming Protocol) no VLC. Nesse modo, o VLC funciona como um servidor RTSP e o cliente se conecta através de uma URL padrão desse protocolo.

Para configurar, iniciei uma nova transmissão no VLC, escolhi um arquivo de vídeo e selecionei RTSP como saída. Em seguida, defini a porta do servidor (usei a porta 8554) e o caminho do vídeo, deixando como /video.

Com isso, o fluxo ficou acessível pelo endereço:

`rtsp://IP_do_servidor:8554/video`

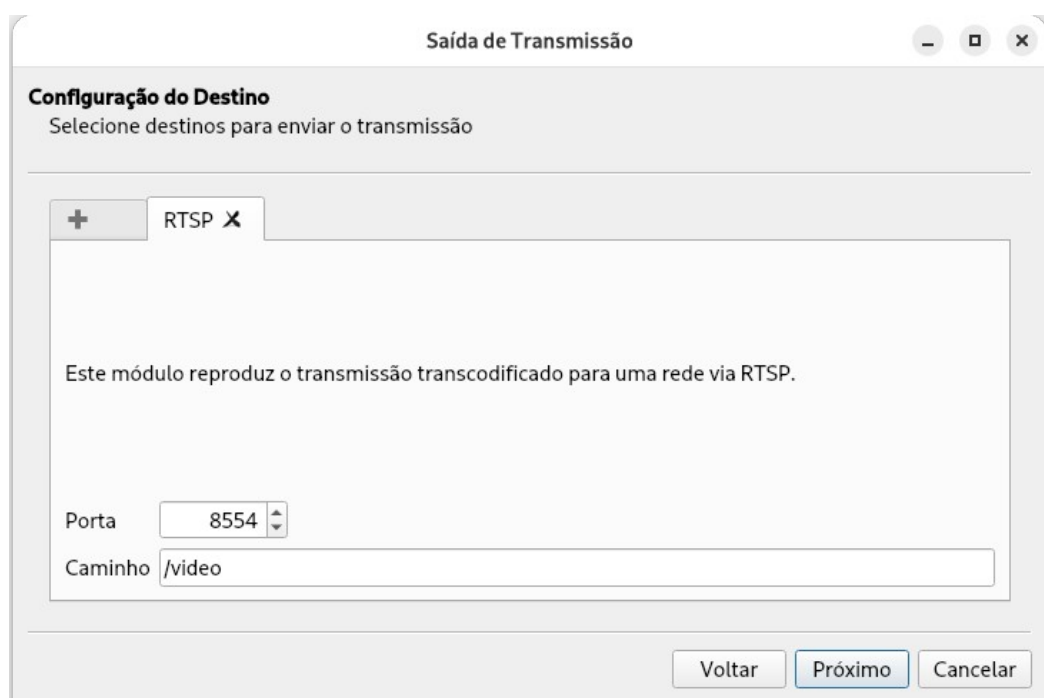


Figura 20: Configuração de destino do streaming via protocolo RTSP na porta 8554

No cliente, abri o VLC e utilizei a opção Abrir Transmissão de Rede, inserindo exatamente a URL configurada. O vídeo carregou normalmente, confirmando que o servidor RTSP estava funcionando.

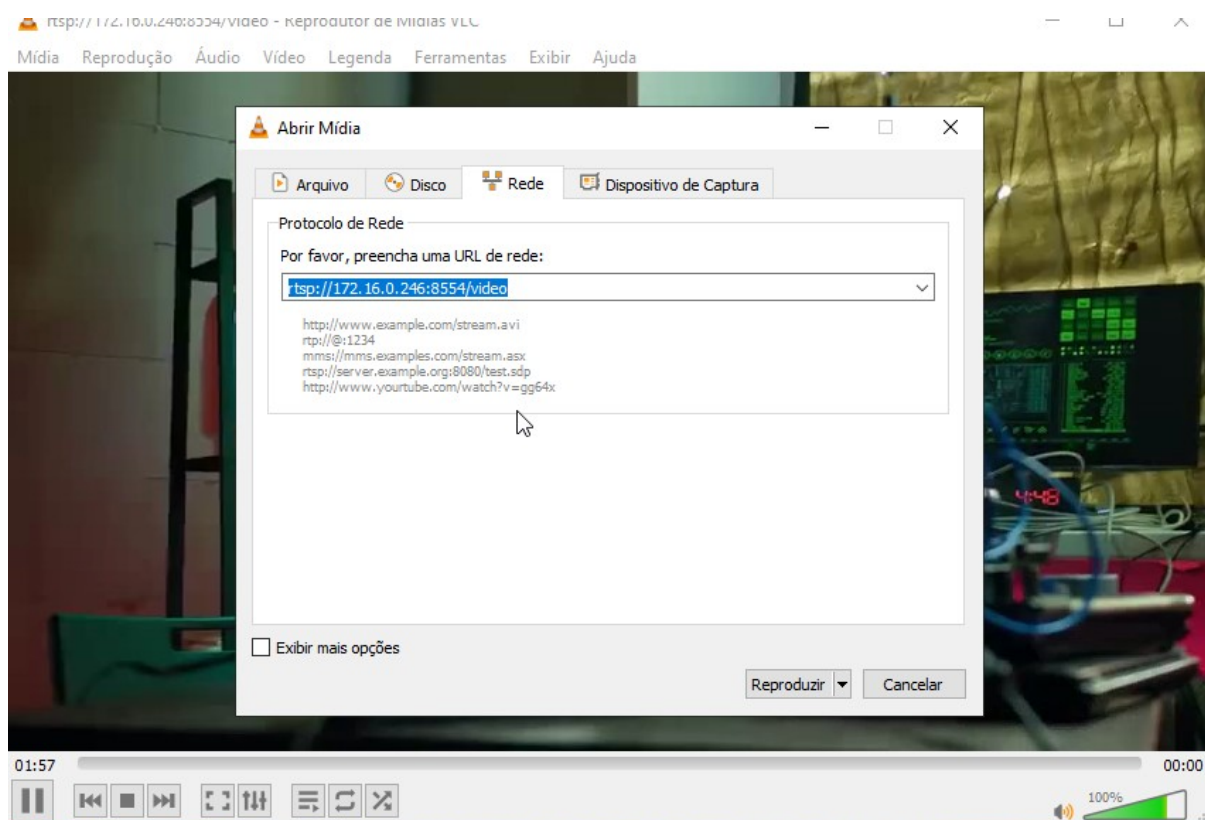


Figura 21: VLC cliente acessando o stream RTSP

Depois de iniciar a transmissão em RTSP, comecei a captura no Wireshark para analisar o tráfego. O filtro que utilizei foi o `rtsp or rtp or rtcp`, pois ele garante que consigo ver tanto os pacotes de controle quanto os pacotes de dados do vídeo na mesma tela.

O print mostra bem o handshake que o professor pediu. O processo de negociação do RTSP segue esta sequência:

- DESCRIBE: O cliente começa pedindo informações sobre o stream.
- SETUP: Aparece em seguida, e é onde o cliente e o servidor negociam em quais portas UDP o vídeo será enviado (RTP/RTCP), que são diferentes da porta TCP de controle 8554.
- PLAY: O cliente envia esse comando para iniciar a transmissão do vídeo.
- TEARDOWN: Ocorre quando o cliente fecha a sessão, encerrando o fluxo.

Logo após o PLAY, é possível ver a aparição dos pacotes de RTP (que carrega o vídeo) e RTCP (que é o protocolo de controle da mídia). Isso confirma que o RTSP só cuida do controle da sessão, enquanto o fluxo de dados do vídeo é entregue pelo RTP em portas separadas.

A captura também mostra várias mensagens ICMP de "Port unreachable" (porta indisponível). Isso é comum e mostra que, apesar do RTSP ter negociado e do servidor ter tentado enviar o fluxo de vídeo (RTP), o cliente não estava conseguindo receber nas portas UDP negociadas, provavelmente por conta de alguma configuração de firewall. No entanto, o objetivo de capturar o handshake foi cumprido e a captura prova o funcionamento dos protocolos.

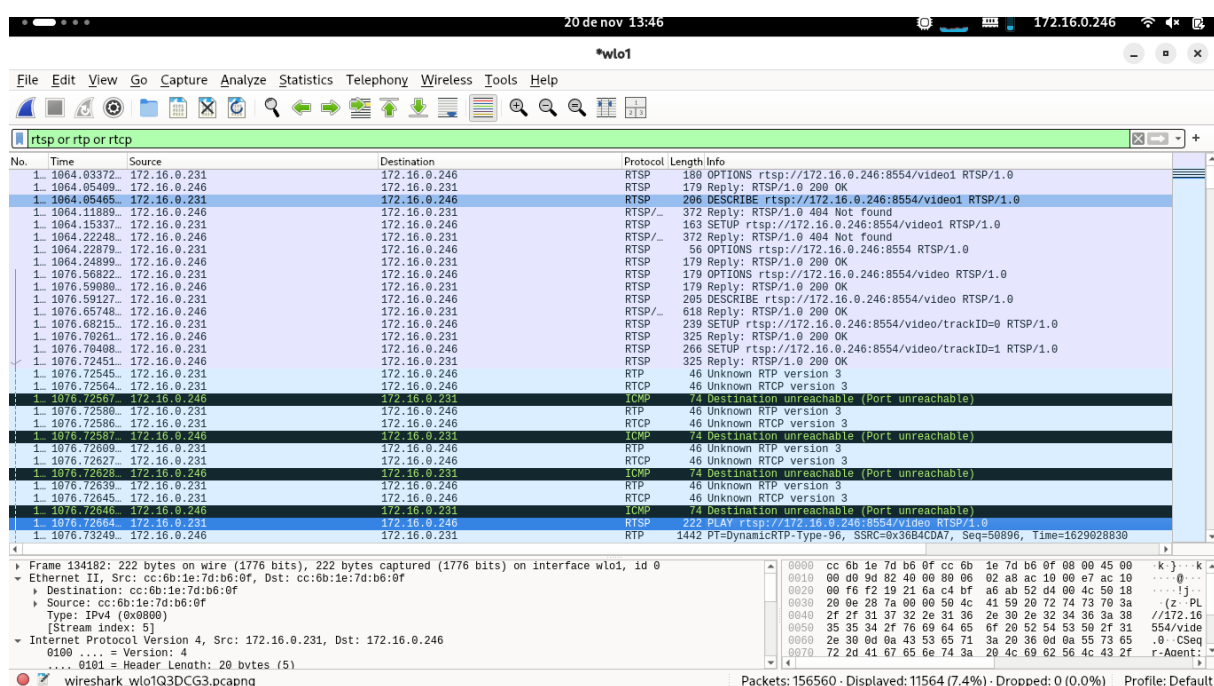


Figura 22: Captura do Wireshark do handshake RTSP

2.5 Transmissão Multicast com OBS Studio

Para finalizar a prática, realizei a tarefa de transmitir a área de trabalho e a câmera utilizando o OBS Studio. Diferente do VLC, que transmite arquivos prontos, o OBS permite capturar fontes em tempo real e oferece um controle profissional sobre a codificação. O objetivo foi configurar uma transmissão em multicast e analisar como a rede e o protocolo UDP se comportam sob estresse.

2.5.1 Configuração da Fonte e Saída no OBS

No OBS, o primeiro passo foi definir o que seria transmitido. Na aba de "Fontes" (Sources), adicionei uma "Captura de Tela" (Display Capture) para transmitir meu monitor e também fiz testes adicionando uma câmera de vídeo.

A configuração mais importante foi a de saída (Output). Para transmitir via multicast sem depender de um servidor externo (como YouTube), mudei o modo de saída para Advanced e o tipo para Custom Output (FFmpeg).

Na URL de saída, configurei o endereço multicast e a porta UDP definidos para o teste: `udp://226.0.0.89:1234?pkt_size=1316`

Também defini o formato do container como mpegts (MPEG Transport Stream) e ajustei o Video Bitrate para 8000 Kbps. Essa taxa alta foi escolhida propositalmente para gerar um fluxo de dados pesado e constante, ideal para testar a capacidade da rede.

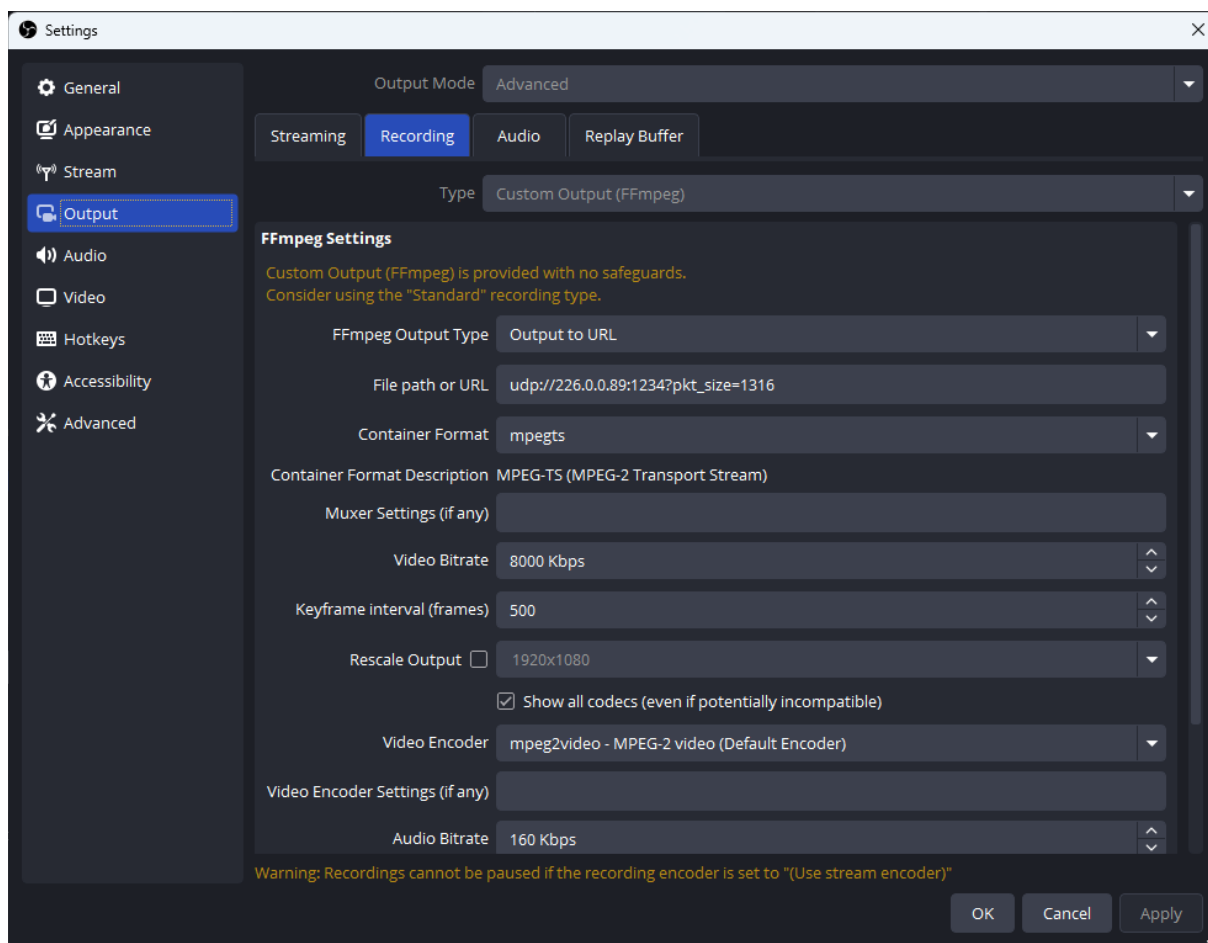


Figura 23: Configuração de saída no OBS

Para que essa transmissão funcionasse no ambiente do laboratório, foi necessário um passo extra que não fizemos no VLC. Como o endereço 226.0.0.89 é de uma rede multicast, o Windows não sabia nativamente para qual interface enviar esses pacotes.

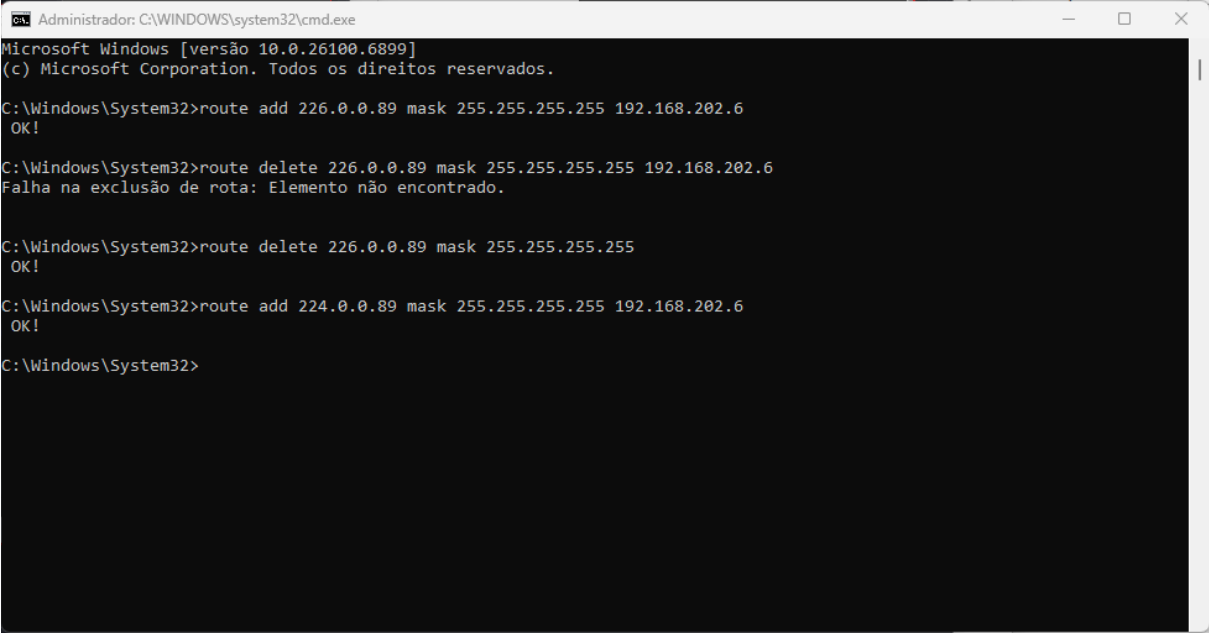
Se eu apenas iniciasse o streaming, os dados não sairiam da minha máquina corretamente. Para resolver isso, precisei adicionar uma rota estática no Windows. Abri o Prompt de Comando (CMD) como administrador e usei o comando `route add`, informando que todo tráfego para 226.0.0.89 deveria ser encaminhado para o gateway da rede (192.168.202.6).

O comando executado foi: `route add 226.0.0.89 mask 255.255.255.255 192.168.202.6`

2.5.2 Configuração de Rotas (Windows e Linux)

Para que o tráfego multicast funcionasse corretamente na rede do laboratório, foi necessário criar rotas estáticas. Como o endereço 226.0.0.89 não pertence à rede local padrão, o sistema operacional não sabia para qual gateway enviar esses pacotes.

Na máquina transmissora (Windows), precisei abrir o terminal como administrador e adicionar uma rota apontando para o gateway 192.168.202.6. O comando utilizado foi: `route add 226.0.0.89 mask 255.255.255.255 192.168.202.6`



```
Administrador: C:\WINDOWS\system32\cmd.exe
Microsoft Windows [versão 10.0.26100.6899]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Windows\System32>route add 226.0.0.89 mask 255.255.255.255 192.168.202.6
OK!

C:\Windows\System32>route delete 226.0.0.89 mask 255.255.255.255 192.168.202.6
Falha na exclusão de rota: Elemento não encontrado.

C:\Windows\System32>route delete 226.0.0.89 mask 255.255.255.255
OK!

C:\Windows\System32>route add 224.0.0.89 mask 255.255.255.255 192.168.202.6
OK!

C:\Windows\System32>
```

Figura 24: Configuração de Rotas no Windows

Também realizei a configuração na máquina cliente que estava rodando Linux (Debian). Para que ela pudesse receber ou enviar pacotes multicast nesse grupo, usei o comando ip route: `ip route add 226.0.0.89/32 via 192.168.202.6`

```
root@debian:~# ip route add 226.0.0.89/32 via 192.168.202.6
root@debian:~# ip route
default via 192.168.200.253 dev eno1 proto dhcp src 192.168.202.179 metric 100
192.168.200.0/22 dev eno1 proto kernel scope link src 192.168.202.179 metric 100
226.0.0.89 via 192.168.202.6 dev eno1
```

Figura 25: Configuração de Rotas no Linux

Sem essas rotas, os pacotes ficariam "presos" na máquina de origem ou seriam descartados pelo gateway errado.

2.5.3 Recepção do Streaming

Para validar a transmissão, configurei o VLC na máquina cliente para abrir o fluxo de rede. Utilizei o mesmo endereço e porta configurados no OBS: `udp://@226.0.0.89:1234`

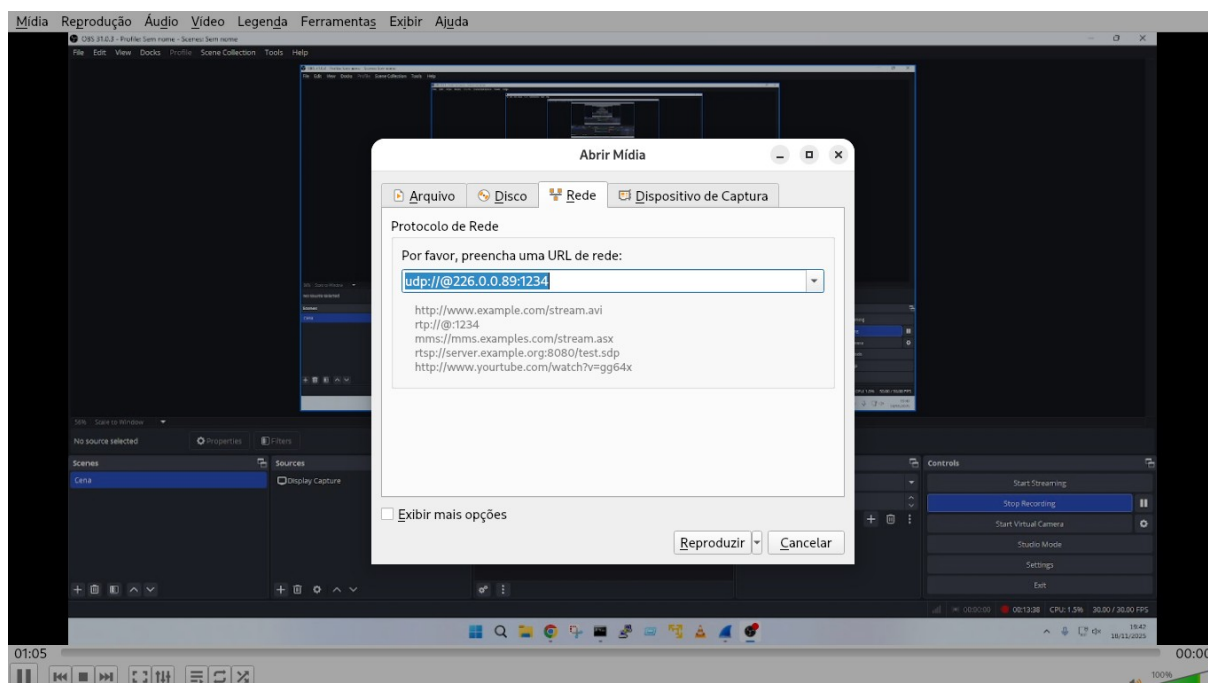


Figura 26: A transmissão funcionou corretamente, exibindo a tela capturada em tempo real.

2.5.4 Simulação de Falhas com Clumsy

Para demonstrar a fragilidade do protocolo UDP, utilizei a ferramenta Clumsy. Configurei a ferramenta para interceptar os pacotes de rede e aplicar duas condições adversas:

1. Lag: Atraso de 50 ms.
2. Drop: Perda de 10% dos pacotes.

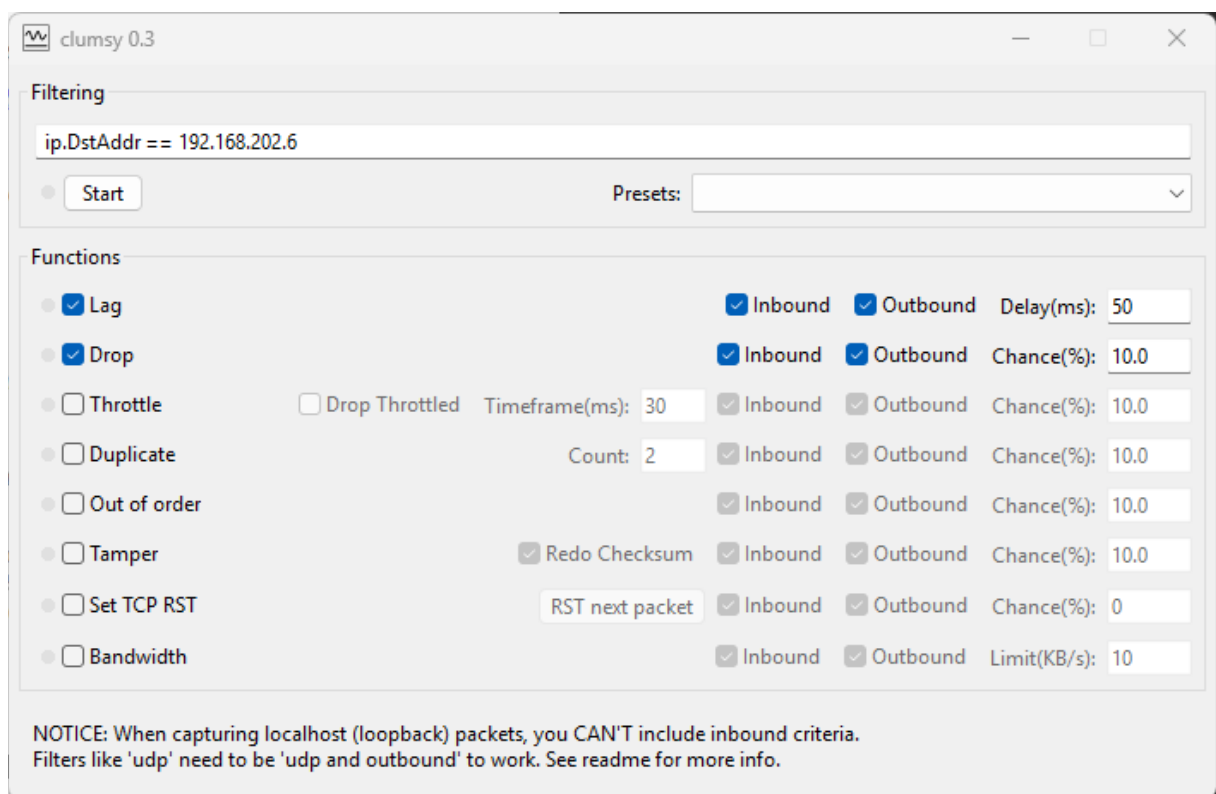


Figura 27: Clumsy configurado para gerar atraso e perda de pacotes na rede

O resultado foi imediato. Assim que a perda de pacotes começou, a imagem recebida no VLC (neste teste, usando a câmera) ficou totalmente distorcida e pixelizada. Isso ocorre porque, no streaming via UDP, não há retransmissão de dados perdidos; o player apenas tenta montar a imagem com os dados que chegaram, gerando falhas visuais.

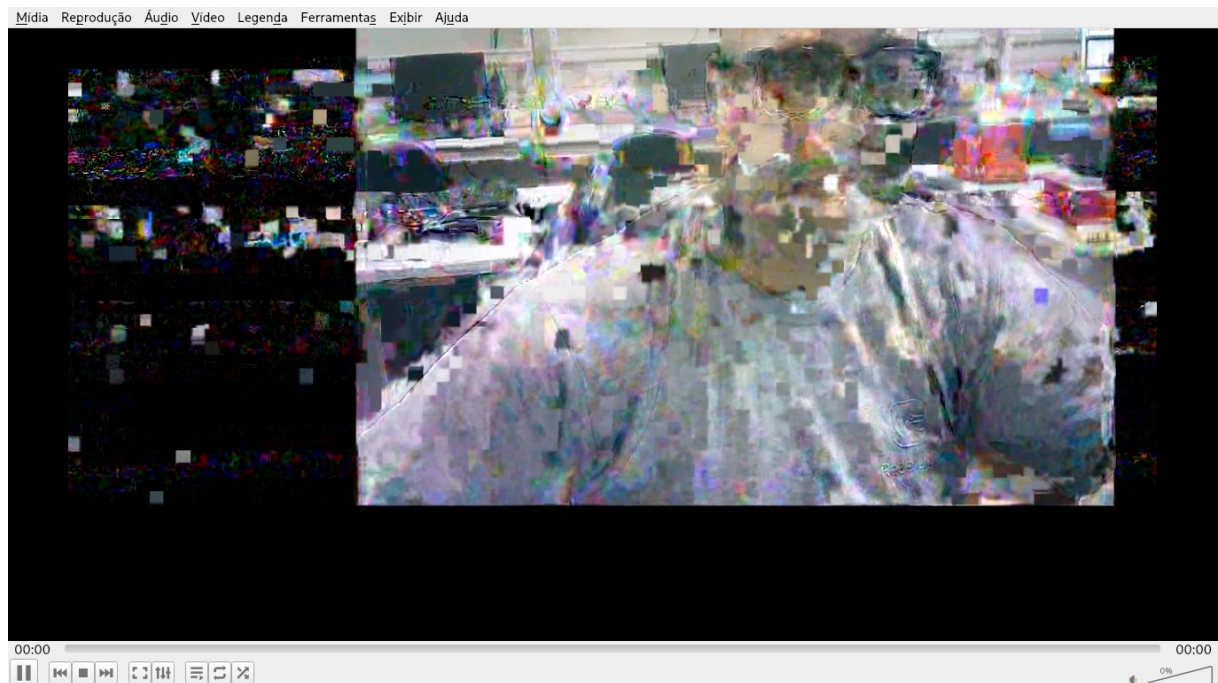


Figura 28: Efeito visual da perda de pacotes UDP: a imagem da câmera falha e distorce

2.5.5 Análise de Tráfego no Wireshark

Por fim, analisei o tráfego gerado pelo OBS utilizando o Wireshark. Abri a captura realizada durante a aula e apliquei o filtro `ip.dst == 226.0.0.89` para isolar apenas os pacotes enviados para o grupo multicast.

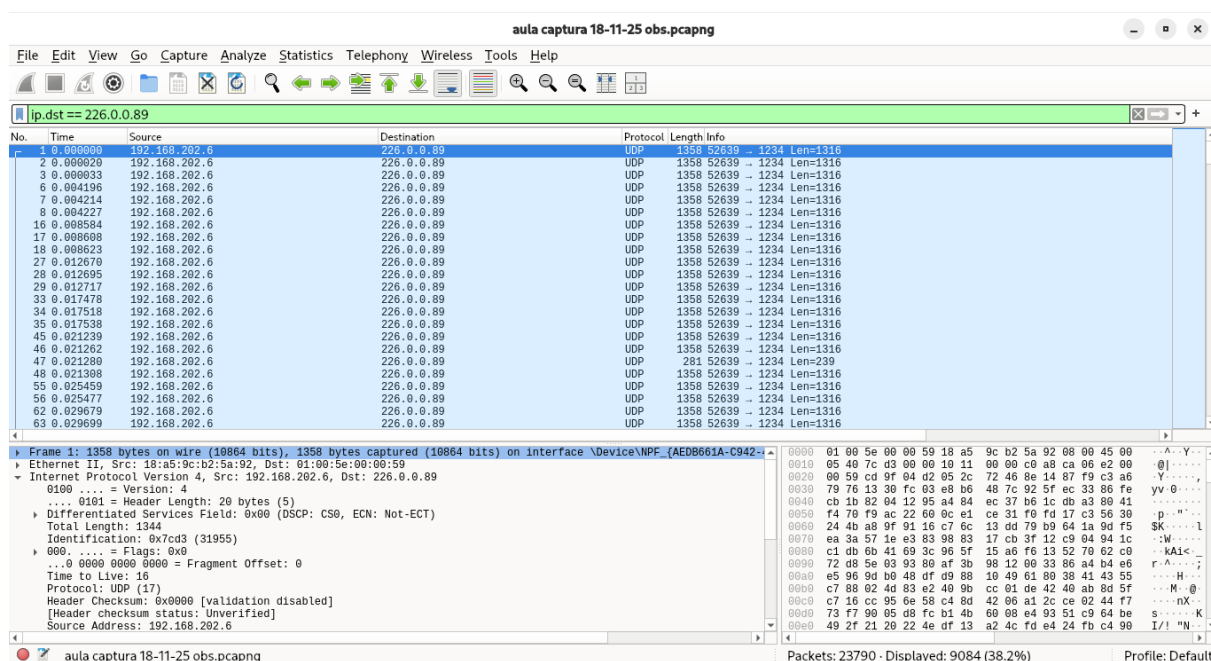


Figura 29: Captura do fluxo UDP do OBS enviado para a porta 1234 do endereço multicast

A captura confirmou que o OBS gera um fluxo intenso e constante de pacotes UDP. O print mostra claramente que os pacotes estão sendo enviados para a porta de destino 1234, exatamente como configurei no OBS. Além disso, o tamanho dos pacotes (Length 1358) indica que eles estão carregados com dados de vídeo. Diferente do RTSP, onde vimos a negociação, aqui o tráfego é direto: assim que o stream inicia, os dados são enviados continuamente para a rede multicast.

3. CONCLUSÃO

A realização desta prática foi fundamental para consolidar o conhecimento sobre redes multimídia, permitindo sair da teoria e entender o que realmente acontece "nos bastidores" durante uma transmissão de vídeo.

Ficou claro que a escolha do modo de transmissão define a eficiência da rede. Na prática, observei que o Unicast é funcional para poucos clientes, mas consome banda repetida, enquanto o Multicast se mostrou a solução ideal para distribuição em escala, embora exija uma configuração de infraestrutura mais atenta — como a criação de rotas estáticas que precisei implementar para que o tráfego fluísse corretamente no laboratório.

A análise com o Wireshark foi o ponto alto do aprendizado técnico. Ao comparar os protocolos, percebi uma diferença crucial de arquitetura:

- No streaming via HTTP, o controle é passivo e "sem estado" (stateless); o servidor apenas entrega os dados via TCP garantindo a ordem, funcionando quase como um download contínuo.
- Já no RTSP, entendi o conceito de um protocolo "com estado" (stateful). Ele atua como um verdadeiro "controle remoto" de rede, mantendo uma sessão ativa e trocando comandos específicos (como SETUP e PLAY) via TCP, enquanto o vídeo flui separadamente via RTP/UDP. Essa separação oferece um gerenciamento muito mais refinado da transmissão.

Por fim, o teste de estresse com o OBS Studio e a ferramenta Clumsy comprovou a natureza do protocolo UDP: ele é rápido e essencial para tempo real, mas não possui recuperação de erros. Ver a imagem da câmera pixelizar e falhar instantaneamente ao simular a perda de pacotes deixou evidente o desafio de manter a qualidade em transmissões ao vivo. Concluo que configurar um serviço de streaming exige não apenas ajustar o encoder, mas garantir que a rede e o protocolo escolhido estejam alinhados com a criticidade da aplicação.

4. REFERÊNCIAS

- Material do servidor Mussum
- Notebooklm